CLOUD COMPUTING — STUDI KASUS

IaaS vs PaaS vs SaaS

Membandingkan tiga model layanan cloud lewat satu aplikasi nyata: **JogjaTrip**, perencana perjalanan wisata Yogyakarta.

[Nama Kelompok]

[Mata Kuliah] · [Tanggal Presentasi]

IaaS PaaS SaaS

KONSEP DASAR

Apa itu Cloud Computing?

Menyewa resource komputasi — server, storage, jaringan — dari penyedia cloud, dibayar sesuai pemakaian, diakses lewat internet. Bedanya tiga model layanan: **siapa yang mengurus apa.**

IaaS

Infrastructure as a Service

Sewa server (VM). Kita install & urus sendiri: OS, web server, database.

PaaS

Platform as a Service

Serahkan kode. Platform mengurus OS, runtime, dan database terkelola.

SaaS

Software as a Service

Tinggal pakai aplikasi jadi lewat browser. Tidak ada yang kita urus.

Kontrol penuh, kita yang repot

Siap pakai, kontrol minim

KONSEP DASAR

5 Karakteristik Cloud Computing

Standar formal **NIST SP 800-145** mendefinisikan cloud computing lewat 5 karakteristik wajib — ini yang nanti membedakan VM lokal demo kita dari IaaS cloud sungguhan.

Karakteristik (NIST)	On-Premises	VM Lokal (Demo)	Cloud Publik Asli
On-demand self-service	Tidak	**Sebagian**	**Ya**
Broad network access	Tidak	Tidak	**Ya**
Resource pooling	Tidak	Tidak	**Ya**
Rapid elasticity	Tidak	Tidak	**Ya**
Measured service	Tidak	Tidak	**Ya**

ANALOGI

Pizza as a Service

Cara paling mudah memahami bedanya: bayangkan empat cara makan pizza.

🏠

ON-PREMISES

Masak sendiri

Beli oven, beli bahan, uleni adonan sendiri di rumah.

🔧

IAAS

Kit pizza beku

Oven dan bahan sudah disediakan. Kita yang panggang dan sajikan.

➤

PAAS

Pesan delivery

Tinggal pesan. Dimasak dan diantar, kita cukup makan.

★

SAAS

Makan di restoran

Duduk, pesan, semuanya sudah disiapkan untuk kita.

KONSEP DASAR

Siapa Mengurus Apa?

Semakin ke kanan tabel, semakin banyak lapisan yang dipindah tanggung jawabnya dari kita ke provider.

Lapisan	On-Premises	IaaS	PaaS	SaaS
Aplikasi & Data	**Anda**	**Anda**	**Anda**	**Provider**
Runtime & Middleware	**Anda**	**Anda**	**Provider**	**Provider**
Sistem Operasi	**Anda**	**Anda**	**Provider**	**Provider**
Virtualisasi	**Anda**	**Provider**	**Provider**	**Provider**
Server, Storage & Jaringan	**Anda**	**Provider**	**Provider**	**Provider**

STUDI KASUS

Mengenal JogjaTrip

Aplikasi perencana perjalanan wisata Kota Yogyakarta. Satu aplikasi ini akan kita deploy dengan tiga model layanan sekaligus.

- ✓ Katalog 72 destinasi, itinerary builder multi-hari + urut otomatis jarak terdekat
- ✓ Rekomendasi & itinerary otomatis berbasis AI (Gemini)
- ✓ Prakiraan cuaca per hari, kolaborasi itinerary, notifikasi pengingat
- ✓ PWA (installable + offline), login dengan Google, dashboard admin

TECH STACK

Laravel 12 · PHP 8.2

MySQL 8

Tailwind + Alpine.js (Vite)

Docker (multi-stage build)

Kode sama, dideploy 3 cara: IaaS (VM manual), PaaS (Railway), dan sudah memakai beberapa SaaS pihak ketiga.

DEMO · SAAS

SaaS pada JogjaTrip

JogjaTrip sudah memakai beberapa layanan SaaS pihak ketiga — kita tidak perlu membangun AI, sistem login, atau layanan cuaca sendiri.

Google Gemini API

Rekomendasi destinasi personalisasi & generator itinerary otomatis.

Google OAuth

Login dengan akun Google, tanpa kita membangun sistem auth sendiri.

Open-Meteo

Prakiraan cuaca per hari untuk tanggal itinerary, gratis tanpa API key.

OpenStreetMap

Peta lokasi destinasi & itinerary lewat Leaflet.js.

★ Setelah di-deploy, JogjaTrip sendiri juga menjadi **SaaS** bagi wisatawan yang memakainya — cukup dibuka lewat browser.

DEMO · IAAS

Arsitektur IaaS: VM Lokal (VirtualBox)

Simulasi IaaS secara lokal — semua komponen kita install & konfigurasi sendiri, persis seperti di cloud asli.



DEMO · IAAS

Langkah-Langkah Deploy

6 tahap, dikerjakan manual lewat SSH ke VM lokal (VirtualBox). Panduan lengkap: **docs/deployment/IAAS-GUIDE.md**.

1. Buat VM Ubuntu di VirtualBox, aktifkan SSH server
2. Setup port forwarding (SSH 2222→22, HTTP 8080→80)
3. SSH masuk, install Nginx, PHP, MySQL, Composer, Node
4. Clone repo, composer install, npm run build
5. Isi .env (localhost:8080), migrate --seed, storage:link
6. Konfigurasi Nginx, pasang cron scheduler

```
PS> ssh -p 2222 user@localhost

sudo apt install -y nginx php-fpm mysql-server
git clone https://github.com/anugrahhhhh/
  tripplane-jogja-itenary.git
composer install --no-dev
npm ci && npm run build
php artisan migrate --seed
crontab -e  # tambah schedule:run
```

DEMO · IAAS

Hasil & Verifikasi

Setelah semua langkah selesai, aplikasi diakses lewat **http://localhost:8080** (port forwarding dari VM lokal).

[TEMPEL SCREENSHOT DEMO IAAS DI SINI]

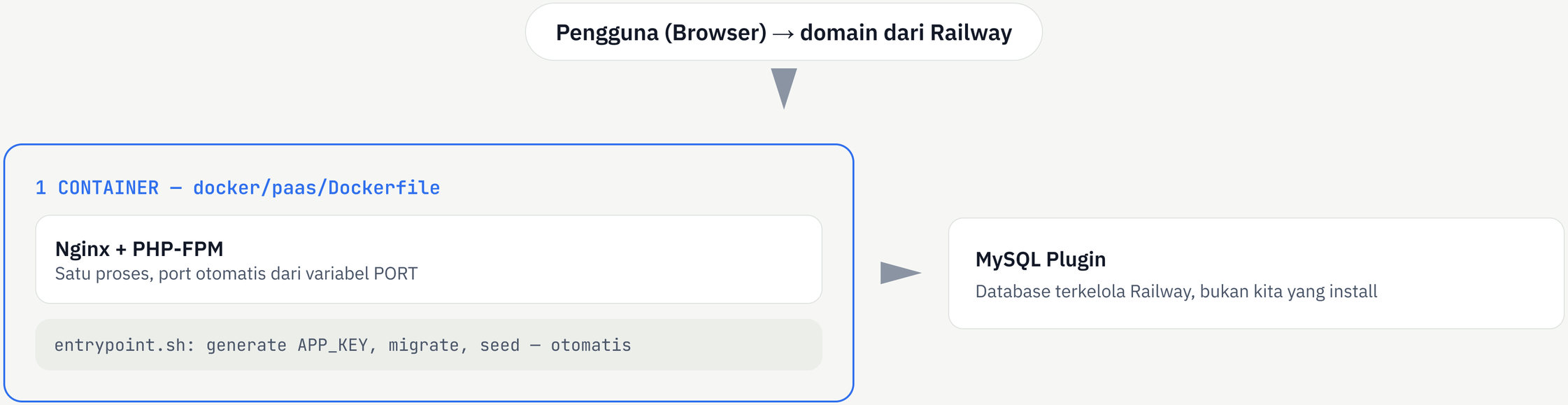
- ✓ Katalog, halaman destinasi, dan login terbuka normal
- ✓ Migrasi & seeding database berhasil (72 destinasi, review, kategori)
- ✓ Cron aktif, pengingat itinerary terjadwal

Waktu setup total: **[isi waktu aktual]**

DEMO · PAAS

Arsitektur PaaS: Railway

Kode yang sama, dibungkus satu Dockerfile. Tidak ada VM, tidak ada SSH.



DEMO · PAAS

Langkah-Langkah Deploy

4 langkah, semuanya lewat dashboard web. Panduan lengkap: **docs/deployment/PAAS-GUIDE.md**.

1. New Project → Deploy from GitHub repo
2. Add service → Database → MySQL
3. Isi environment variable (API key, referensi DB)
4. Railway build & deploy otomatis — aplikasi online

Variables

```
DB_HOST=${{MySQL.MYSQLHOST}}
DB_PORT=${{MySQL.MYSQLPORT}}
DB_DATABASE=${{MySQL.MYSQLDATABASE}}
DB_USERNAME=${{MySQL.MYSQLUSER}}
DB_PASSWORD=${{MySQL.MYSQLPASSWORD}}
GEMINI_API_KEY=...
APP_URL=${{RAILWAY_PUBLIC_DOMAIN}}
```

DEMO · PAAS

Praktis, Tapi Ada Trade-off

Kemudahan PaaS datang dengan keterbatasan yang baru terasa setelah dicoba sendiri.

⚠

Storage tidak permanen

Foto destinasi yang diupload admin disimpan di dalam container. Setiap redeploy, container lama dibuang — foto **hilang**. Filesystem container bersifat ephemeral.

🕑

Scheduler ikut platform

Tidak ada crontab sendiri. Pengingat itinerary harus dijalankan lewat proses long-running terpisah atau fitur cron milik Railway.

🔒 Dan yang paling penting: **kita tidak pernah menyentuh OS** — tidak ada SSH, tidak ada firewall untuk diatur.

DEMO · ANALISIS

Keamanan: Siapa Patch Apa?

Model tanggung jawab yang sama juga berlaku untuk keamanan — permukaan serangan (attack surface) berbeda tergantung siapa yang bertanggung jawab atas tiap lapisan.

Aspek Keamanan	IaaS (VM Lokal)	PaaS (Railway)
Patch OS & kernel	**Kita, manual (apt upgrade)**	**Provider, otomatis**
Patch runtime (PHP-FPM, Node.js)	**Kita, manual**	**Provider, lewat rebuild image**
Exposure ke internet publik	**Tidak ada (hanya localhost)**	**Publik, langsung bisa diakses**
Isolasi antar aplikasi/tenant	**Tidak relevan, VM milik sendiri**	**Provider jamin isolasi container**
Backup & disaster recovery	**Tanggung jawab kita penuh**	**Tergantung provider, tidak otomatis**
Respon terhadap kerentanan (CVE) baru	**Kita pantau & patch sendiri**	**Provider, untuk layer di bawah kode**

PERBANDINGAN & KESIMPULAN

Hasil Demo: IaaS vs PaaS

Deploy aplikasi JogjaTrip yang sama, dua cara berbeda.

Aspek	IaaS (VM Lokal)	PaaS (Railway)
Jumlah langkah utama	6 tahap, puluhan command manual	4 langkah lewat dashboard
Waktu setup	**[isi waktu aktual demo]**	**[isi waktu aktual demo]**
Siapa urus OS & server	**Kita (manual lewat SSH)**	**Provider (otomatis)**
Kontrol	Tinggi — bebas ubah apa saja	Rendah — ikut batasan platform
Biaya (untuk demo)	Gratis (hardware sendiri)	Free tier Railway
Masalah yang ditemukan	Azure/Oracle sign-up terblokir (MFA/rate-limit) → pindah VM lokal	Foto hilang saat redeploy (storage)

PERBANDINGAN & KESIMPULAN

Data Nyata dari Proses Deploy

Bukan simulasi di atas kertas — angka-angka ini diambil langsung dari log terminal saat demo IaaS benar-benar berjalan.

150
Paket npm ter-install (`npm ci`)

14
Migrasi database dijalankan

72
Destinasi ter-seed otomatis

2.12s
Waktu build Vite (59 modules)

```
Versi nyata yang terpasang: PHP-FPM 8.5 · Ubuntu 26.04.1 LTS · Node.js v22.22.1 · MySQL 8
```

⚠ `npm audit` menemukan **6 kerentanan** (1 moderate, 3 high, 2 critical) di dependency JS — temuan nyata yang layak didiskusikan, bukan diabaikan begitu saja.

PERBANDINGAN & KESIMPULAN

Kapan Pakai Apa?

IaaS

+ Kontrol penuh, bebas konfigurasi apapun
+ Cocok untuk belajar cara kerja server

– Semua instalasi & keamanan tanggung jawab kita
– Setup paling lama & paling rawan salah

Pakai saat: butuh kontrol penuh / belajar infra

PaaS

+ Deploy cepat, tanpa urus OS
+ Scaling & database terkelola otomatis

– Storage ephemeral, kontrol terbatas
– Biaya bisa naik seiring trafik

Pakai saat: mau fokus ke kode, bukan infra

SaaS

+ Siap pakai, nol instalasi
+ Selalu versi terbaru dari vendor

– Hampir tanpa kustomisasi
– Tergantung penuh pada vendor

Pakai saat: kebutuhan umum, bukan unik

KESIMPULAN

JogjaTrip membuktikan: satu aplikasi, tiga cara berbeda untuk online.

SaaS sudah dipakai

Gemini, Google OAuth, Open-Meteo, OpenStreetMap — tanpa bangun dari nol.

IaaS = kontrol penuh

Kita urus semua sendiri di VM — paling fleksibel, paling berisiko salah.

PaaS = paling praktis

Deploy dalam menit lewat Railway, dengan trade-off storage & kontrol.

REFERENSI

Repo JogjaTrip — github.com/anugrahhhhh/tripplane-jogja-itenary

docs/deployment/IAAS-GUIDE.md & PAAS-GUIDE.md (panduan lengkap di repo)

Microsoft Azure — Shared Responsibility Model

Dokumentasi Railway, Azure for Students, Open-Meteo, Google Cloud Console

Terima Kasih

Pertanyaan & diskusi